

CMPU3010 Databases 2 — Week 2 Lab

SQL Refresher and Introduction to Advanced SQL

This lab will refresh some SQL from Databases 1 and then introduce a type of query that we will investigate further in Wednesday's class.

You are **not expected to remember all SQL syntax from Databases 1**. Hints and examples are provided throughout the lab.

SQL Quick Reference

You may find these patterns useful.

Selecting and sorting <pre>SELECT column1, column2 FROM table WHERE condition ORDER BY column1;</pre>	Joining tables <pre>SELECT ... FROM table1 t1 JOIN table2 t2 ON t1.id = t2.id;</pre>
LEFT Join <pre>SELECT ...FROM table1 t1LEFT JOIN table2 t2 ON t1.id = t2.id;</pre> Returns all rows from table1, including rows with no match in table2.	RIGHT Join <pre>SELECT ...FROM table1 t1RIGHT JOIN table2 t2 ON t1.id = t2.id;</pre> Returns all rows from table2, including rows with no match in table1.
Counting rows <pre>SELECT column1, COUNT(*) AS total FROM table GROUP BY column1;</pre>	Filtering grouped results <pre>SELECT column1, COUNT(*) AS total FROM table GROUP BY column1 HAVING COUNT(*) > 5;</pre>
Subquery <pre>SELECT ... FROM table WHERE column1 > (SELECT AVG(column1) FROM table);</pre>	Correlated Subquery <pre>SELECT s1.service_name, s1.standard_fee, s1.centre_id FROM service s1 WHERE s1.standard_fee > (SELECT AVG(s2.standard_fee) FROM service s2 WHERE s2.centre_id = s1.centre_id);</pre>

Part A — Getting Started

Start by telling PostgreSQL that we are working with the CommunityCare schema:

```
SET search_path TO communitycare;
```

A1. Look at the data

Run:

```
SELECT *  
FROM appointment  
LIMIT 10;
```

Look at the columns in the appointment table. Identify the column that connects an appointment to a patient.

A2. A simple query

Display the:

- appointment_id
- patient_id
- appointment_date
- status

for all **Completed** appointments.

Start with:

```
SELECT  
    appointment_id,  
    patient_id,  
    appointment_date,  
    status  
FROM appointment  
WHERE you complete this part  
ORDER BY appointment_date;
```

- Fill in the missing condition.
 - Remember you use single quotes to enclose strings ('');

A3. Formatting Dates (OPTIONAL)

- Change the format of the date in the output to use the form dd-mon-yyyy.
- Modify the query you write for A2 to use the TO_CHAR function. If you need help check out this page: https://neon.com/postgresql/string-functions/to_char

Part B — Joining Tables Refresher

An appointment contains a `patient_id`, but CommunityCare normally wants to display the patient's **name** rather than their ID. Similarly, CommunityCare would like to know the name of the service associated with each appointment. We therefore need to join to appointment via the foreign keys to other tables to retrieve this information

B1. Appointment and patient details

- Patient name is in the patient table – this is connected to appointment via the foreign key `patient_id` in the appointment table.
- Complete the query:

```
SELECT
    a.appointment_id,
    p.first_name,
    p.last_name,
    a.appointment_date,
    a.status
FROM appointment a
JOIN patient p
    ON you complete this part (a.? = p.?)
ORDER BY a.appointment_date;
```

B2. Add the service

- The service name is in the `service` table. The appointment table also contains a `service_id`, a foreign key to the service table.
- Modify the query you wrote for B1 so that it also joins the service table and displays the **service name**. Your query will now need:

```
FROM appointment a
JOIN patient p
    ON ...
JOIN service s
    ON ...
```

- Your result should contain: Patient | Service | Date | Status

B3. Filter the result

- Modify the query you wrote for B2 so that it displays **Completed appointments only**.
- Use a `WHERE` clause.

B4. Patients with no appointments

CommunityCare would like to identify patients who **have not yet had an appointment**. A normal JOIN only returns patients who have a matching row in appointment. To include patients who do **not** have a matching appointment, we can use a LEFT JOIN.

- Run:

```
SELECT
    p.patient_id,
    p.first_name,
    p.last_name,
    a.appointment_id
FROM patient p
LEFT JOIN appointment a
    ON p.patient_id = a.patient_id
ORDER BY
    p.last_name,
    p.first_name;
```

- Examine the results.
- Consider:
 1. Are patients with no appointments included in the result?
 2. What value appears in appointment_id for a patient who has no appointment?
 3. What would happen if you changed LEFT JOIN to JOIN?
- Now modify the query so that it displays **only patients who have no appointments**.
Hint: Patients with no matching appointment will have NULL in the appointment columns. Use IS NULL to identify these rows.

B5. Services with no bookings (OPTIONAL)

CommunityCare would like to see **all services**, including services that have never been booked.

- Insert the following service:

```
INSERT INTO service
    (service_name, duration_minutes, standard_fee)
VALUES
    ('Speech and Language Therapy', 60, 70.00);
```

- Run:

```
SELECT
    s.service_id,
    s.service_name,
    a.appointment_id
FROM appointment a
RIGHT JOIN service s
    ON a.service_id = s.service_id
ORDER BY s.service_name;
```

- Consider:
 1. Are services that have never been booked included in the result?
 2. What value appears in appointment_id for a service with no appointments?
 3. Why does RIGHT JOIN ensure that all services are included?
 4. Which table is the **right table** in this query?
- Now modify the query so that it displays **only services that have never been booked**.
Hint: When a service has no matching appointment, the columns from appointment will contain NULL. Use `IS NULL` in the WHERE clause to identify these rows

Key point

LEFT JOIN keeps **all rows from the table on the left** of the join.

RIGHT JOIN keeps **all rows from the table on the right** of the join.

Part C — Aggregating Data Refresher

CommunityCare wants to know how frequently each service is being used.

C1. Count appointments

- Start by running:

```
SELECT
    service_id,
    COUNT(*) AS number_of_appointments
FROM appointment
GROUP BY service_id;
```

- What does GROUP BY do here?

C2. Display the service name

- A report containing only `service_id` is not very useful.
- Modify the query for C1 so that it joins service and displays:

Service Number of appointments

... ..

Hint: You will need more than one column in the GROUP BY clause. If you need a refresher on GROUP BY check this site: <https://www.datacamp.com/doc/postgresql/group-by>

- Amend your query to **order** the results so that the most frequently booked service appears first.

C3. Filter grouped results

- Modify the query you wrote for C2 so that it only displays services with **more than 5 appointments**.

Remember: WHERE filters rows **before** grouping. HAVING filters the groups produced by GROUP BY.

You will need something like: `HAVING ? > 5` (you decide what ? is)

PART D – Subquery Refresher

CommunityCare wants to identify services that cost more than the average service fee.

- First run:

```
SELECT AVG(standard_fee)
FROM service;
```

- Now use the previous query as a subquery:

```
SELECT
    service_name,
    standard_fee
FROM service
WHERE standard_fee > (
    SELECT AVG(standard_fee)
    FROM service
)
ORDER BY standard_fee DESC;
```

- Consider:
 1. What value does the inner query (the second select statement) return?
 2. How is this value used by the outer query?
 3. Does the inner query need any information from the outer query to calculate the average?

In this exercise the inner query can run independently of the outer query.

Part E — A More Difficult Question

Now consider a different type of question. CommunityCare wants to show a patient's appointment history. For every appointment, it wants:

Patient	Appointment date	Previous appointment
Patient A	10 Feb	NULL
Patient A	3 Mar	10 Feb
Patient A	21 Apr	3 Mar
Patient B	14 Feb	NULL
Patient B	8 May	14 Feb

For each appointment we need to ask: **What was the most recent appointment for this patient before the current appointment?**

E1. Break the problem down

- Suppose there is a current appointment: Patient 5 on 21 April
- We want to find appointments where:

`patient_id = 5 and appointment_date < 21 April`

- There might be several earlier appointments. Which aggregate function could give us the **latest** of those dates? `MIN(...)` `MAX(...)` `AVG(...)` `COUNT(...)`
- Conceptually we want: `MAX(appointment_date)` from the appointments belonging to the same patient that occurred before the current appointment.

E2. Try the query

- Run:

```
SELECT
  a.patient_id,
  a.appointment_date,
  (
    SELECT MAX(previous.appointment_date)
    FROM appointment previous
    WHERE previous.patient_id = a.patient_id
      AND previous.appointment_date < a.appointment_date
  ) AS previous_appointment
FROM appointment a
ORDER BY
  a.patient_id,
  a.appointment_date;
```

- Don't worry if you would not have written this query yourself. Try to understand it.
- This is a **correlated subquery**: the inner query **depends on the outer query**. It refers to a value from the current row of the outer query.
- For this line: `previous.patient_id = a.patient_id` consider:
 - Why must the previous appointment belong to the same patient?
- For: `previous.appointment_date < a.appointment_date` consider:
 - Why are we only considering earlier appointments
- For: `MAX(previous.appointment_date)` consider:
 - Why do we want the maximum of the earlier dates?

- Some results will show NULL — this is correct. **These are appointments for which there is no earlier appointment for the same patient.** In these cases, MAX() is calculated over no matching rows and returns NULL.

Part F — Is There a Better Way?

The previous query works. But SQL provides another way to answer questions such as:

- What was the **previous** row?
- What is the **next** row?
- What number is this row within a group?
- How does this row **rank** compared with others?
- What is the **running total** up to this row?

These are examples of problems that can be solved using **window functions**.

F1. Number each patient's appointments

ROW_NUMBER() is a **window function** that assigns a sequential number to each row in the result.

- Run:

```
SELECT
    patient_id,
    appointment_date,

    ROW_NUMBER() OVER (
        PARTITION BY patient_id
        ORDER BY appointment_date
    ) AS appointment_number
FROM appointment
ORDER BY
    patient_id,
    appointment_date;
```

- Look at the result. What happens to the appointment number when the patient_id changes?
- Now remove: PARTITION BY patient_id and run it again. What changes?
- **ROW_NUMBER()** — gives each row a number: 1, 2, 3, ...
- **PARTITION BY patient_id** — creates a separate group for each patient. The numbering restarts at 1 for each patient.
- **ORDER BY appointment_date** — determines the order in which each patient's appointments are numbered.

F2. Previous appointment

LAG() is a **window function** that lets you access a value from a **previous row** without having to join the table to itself or use a correlated subquery.

- Now run:

```
SELECT
    patient_id,
    appointment_date,

    LAG(appointment_date) OVER (
        PARTITION BY patient_id
        ORDER BY appointment_date
    ) AS previous_appointment
FROM appointment
ORDER BY
    patient_id,
    appointment_date;
```

- Compare this with the query FROM Part E.
- Both queries answer essentially the same question.
- Consider:
 1. What does LAG() appear to return?
 2. What does PARTITION BY patient_id appear to do?
 3. Why do we need ORDER BY appointment_date?
 4. Why does the first appointment for each patient have NULL as its previous appointment?
 5. Which query is easier to understand?

LAG(appointment_date) — returns the appointment_date from the previous row.

PARTITION BY patient_id — creates a separate group for each patient. LAG() will only look at appointments belonging to the **same patient**.

ORDER BY appointment_date — puts each patient's appointments into date order, so PostgreSQL knows what "previous" means.

F3. Ranking Services

CommunityCare would like to see which services are most frequently booked.

- First add some additional appointment data:

```
INSERT INTO appointment
(patient_id, staff_id, service_id,
 appointment_date, appointment_time, status)
VALUES
(11, 10, 6, '2026-05-12', '09:00', 'Completed'),
(12, 10, 6, '2026-05-13', '10:00', 'Completed'),
(13, 10, 6, '2026-05-14', '11:00', 'Completed');
```

- Now run this query to count the number of appointments for each service:

```
SELECT
    s.service_name,
    COUNT(a.appointment_id) AS number_of_appointments
FROM service s
LEFT JOIN appointment a
    ON s.service_id = a.service_id
GROUP BY
    s.service_id,
    s.service_name
ORDER BY
    number_of_appointments DESC;
```

- Examine the results.
- We can see the services in order of popularity, but CommunityCare would also like each service to have a **ranking**:

Service	Number of appointments	Rank
Service A	12	1
Service B	10	2
Service C	10	2
Service D	7	4

- Notice that if two services have the same number of appointments, they receive the **same rank**.

- Now you are going to use RANK(). RANK() is a **window function** that assigns a ranking to rows based on a specified order.
- Run:

```
SELECT
    s.service_name,
    COUNT(a.appointment_id) AS number_of_appointments,
    RANK() OVER (
        ORDER BY COUNT(a.appointment_id) DESC
    ) AS service_rank
FROM service s
LEFT JOIN appointment a
    ON s.service_id = a.service_id
GROUP BY
    s.service_id,
    s.service_name
ORDER BY
    service_rank;
```

- Examine the result.
- Try to answer the following:
 1. What determines the rank?
 2. What happens when two services have the same number of appointments?
- **RANK()** — assigns each service a position: 1st, 2nd, 3rd, etc.
- **ORDER BY COUNT(a.appointment_id) DESC** — ranks services according to their number of appointments, from **highest to lowest**.
- The GROUP BY first produces **one row per service** with its appointment count. RANK() then ranks those resulting rows.

- Replace: RANK() with ROW_NUMBER() and run the query again.
 - **What changes? What happens when two services have the same number of appointments?**

Example Output using ROW_NUMBER()		
service_name	number_of_appointments	service_rank
Physiotherapy	15	1
General Consultation	10	2
Counselling	9	3
Health Screening	6	4
Dietitian	6	5
Occupational Therapy	5	6
Example Output using RANK()		
service_name	number_of_appointments	service_rank
Physiotherapy	15	1
General Consultation	10	2
Counselling	9	3
Health Screening	6	4
Dietitian	6	4
Occupational Therapy	5	6

Why does RANK() jump from 4 to 6 in this example? (your result may differ)

Notice: ROW_NUMBER() gives every row a different number. RANK() gives services with the same number of appointments the same rank. This causes the next rank to be skipped.

End of Lab - What Have We Done?

You started with SQL that should be familiar from **Databases 1**:

SELECT
↓
WHERE
↓
JOIN
↓
GROUP BY
↓
COUNT
↓
HAVING

You then encountered questions that require us to consider a row **in relation to other rows**, for example:

- **What number is this appointment in a patient's appointment history?**
- **How does a service rank compared with other services?**
- **What was the patient's previous appointment?**

You explored three **window functions** that make these types of questions easier:

ROW_NUMBER() → Number rows within a group

RANK() → Rank rows based on a value

LAG() → Access a value from the previous row

For the previous appointment problem, you also saw two different approaches:

DB1 approach: Correlated subquery + MAX()



DB2 approach: LAG() + OVER()

This introduces an important **Databases 2** topic: **Window Functions**

On Wednesday we will look more closely at:

- OVER(), PARTITION BY and ORDER BY
- ROW_NUMBER() and RANK()
- LAG() and LEAD()
- aggregate window functions such as SUM(), AVG() and COUNT()

and use them to answer more complex questions about the **CommunityCare** data.